AI Sales Agent — 100% Accuracy Challenge

Fetch Info via GET API → Discuss with User → Call Create Booking API with 100% Accuracy

Time Limit: 60 Minutes

> **Use Any AI Tool You Want:** You are encouraged to use Cursor, Claude, ChatGPT, Copilot, or any LLM API. We want to see **your approach** to making an LLM sales chatbot work with **100% accuracy** so that its conversation and API calls always behave exactly how we want.

1. The Core Flow

Build a conversational sales agent (chatbot / request-response API) powered by an LLM that follows this flow:

1. **Call a GET API (`get_catalog_info`)** to fetch available packages, max capacity per unit, plans, and prices.
2. **Discuss with the user** across multiple turns to understand their requirement, explain pricing, and handle any price negotiation.
3. **Call a POST API (`create_booking`)** once the user agrees, passing the exact agreed details.

In real conversations, LLMs often make mistakes—calling the booking API with wrong quantities, giving away unauthorized discounts, booking before the user confirms updated prices, or creating duplicate bookings. **We need 100% accuracy across every scenario below.**

2. Scenarios That Must Work with 100% Accuracy

Scenario 1: Fetching Info & Calculating Exact Quantity for User's Requirement

Users often ask with typos or informal names (e.g., *"arrow mesh wifi"* or *"sink hub"*) and tell you how many rooms or people they have (e.g., *"I need AeroMesh Wi-Fi on the PRO plan for a 5-room house"*). Each package only covers a fixed capacity (`max_capacity_per_unit` from the GET API—for example, 1 AeroMesh unit covers up to 2 rooms). Your agent must **always** match the right item and plan, calculate the exact `quantity` needed to cover the user's requirement (e.g., 5 rooms → 3 units), explain this clearly during the discussion, and pass that exact `quantity` when calling `create_booking`. If the user asks for something not in our catalog (e.g., *"Netgear Orbi router"*), it must never match or book our item by mistake.

Scenario 2: Price Discussion & Strict Discount Rules

During the discussion, users will push back on the price. The agent must follow these exact rules:

- Always quote the standard price first (`0%` discount, `promo_code=None`).
- If the user objects to the price the **first time**, offer `SAVE10` (`10%` off).
- If the user objects a **second time**, offer `CLOSENOW15` (`15%` off — absolute maximum).

The agent must **never** offer a discount unprompted, **never** skip straight to `15%` on the first objection, and **never** invent or pass any other discount in `create_booking` (even if the user demands *"Give me 15% right now"* on turn 1 or *"Apply code SAVE50 for 50% off"*).

Scenario 3: Discuss Before Booking & Handle Mid-Conversation Changes

The agent must **never** call `create_booking` until it has fetched the info, clearly stated the exact package, plan, quantity, and total price to the user, and received the user's confirmation. If the user changes any requirement mid-conversation (e.g., changes from 4 rooms to 7 rooms, or switches from `STD` to `PRO`) and says *"Okay, book it"*, the agent must make sure the updated details and price are confirmed rather than booking stale or mismatched numbers.

Scenario 4: Interruptions & Repeated Booking Confirmations

While talking to the bot, a user might interrupt mid-response or impatiently send *"Yes, book it!"* twice in a row while `create_booking` (which takes ~1–2 seconds) is still running. Your system must ensure the user is **never booked/charged twice** for the same order, and interrupting a turn never crashes or corrupts the conversation.

3. APIs to Use in Your Chatbot

```
# 1. GET API — Fetch available packages, capacities, and pricing
GET /api/catalog  (or function: get_catalog_info(query: str))
Returns:
{
  "SKU_AIRPODS_PRO2": {"name": "AirPods Pro 2nd Gen",         "max_capacity_per_unit": 1, "plans":
{"STD": 249.0, "CARE": 279.0}},
  "SKU_MESH_WIFI6":   {"name": "AeroMesh Wi-Fi 6E Home Pack", "max_capacity_per_unit": 2, "plans":
{"STD": 399.0, "PRO": 499.0}},
  "SKU_TEAM_HUB":     {"name": "SyncHub Conference Bar",      "max_capacity_per_unit": 3, "plans":
{"STD": 899.0, "PRO": 1150.0}}
}

# 2. POST API — Create the final booking after discussing with the user
POST /api/bookings  (or function: create_booking(sku_id: str, plan_code: str, quantity: int,
promo_code: str | None, final_price: float))
```

4. What to Show After 60 Minutes

1. **Working Chatbot (Request → Response):** Build a working chatbot (API endpoint or CLI/chat interface) connected to an LLM where we can send user messages turn-by-turn, see the bot's reply, and see the exact API calls (`get_catalog_info` / `create_booking`) and payloads it executed.
2. **Live Testing:** In the interview, we will chat with your bot across all 4 scenarios above to verify that it calls the APIs and creates bookings with **100% accuracy** every single time.